

I have used Kaggle Dataset of IIT Bombay English to Hindi Translation. This file contains parallel sentences in English and Hindi.

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.layers import Input, LSTM, Embedding, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load the dataset
df = pd.read_csv('/content/hindi_english_parallel.csv')
df = df[['english', 'hindi']].dropna().iloc[:10000] # Using 10,000 samples

english_texts = df['english'].values
hindi_texts = ['\t' + text + '\n' for text in df['hindi'].values] # Adding SOS (\t) and EOS (\n) tokens

# Tokenization
eng_tokenizer = Tokenizer(char_level=True)
eng_tokenizer.fit_on_texts(english_texts)
eng_vocab_size = len(eng_tokenizer.word_index) + 1

hin_tokenizer = Tokenizer(char_level=True)
hin_tokenizer.fit_on_texts(hindi_texts)
hin_vocab_size = len(hin_tokenizer.word_index) + 1

# Convert to sequences
encoder_input_data = eng_tokenizer.texts_to_sequences(english_texts)
decoder_input_data = hin_tokenizer.texts_to_sequences(hindi_texts)

# Padding sequences
max_encoder_seq_length = max([len(txt) for txt in encoder_input_data])
max_decoder_seq_length = max([len(txt) for txt in decoder_input_data])

encoder_input_data = pad_sequences(encoder_input_data, maxlen=max_encoder_seq_length, padding='post')
decoder_input_data = pad_sequences(decoder_input_data, maxlen=max_decoder_seq_length, padding='post')

# Prepare decoder target data
decoder_target_data = np.zeros((len(english_texts), max_decoder_seq_length, hin_vocab_size), dtype="float32")

for i, seq in enumerate(decoder_input_data):
    for t, word_idx in enumerate(seq):
        if t > 0:
            decoder_target_data[i, t - 1, word_idx] = 1.0

latent_dim = 256 # Size of LSTM units

# Encoder
encoder_inputs = Input(shape=(None,))
encoder_embedding = Embedding(eng_vocab_size, latent_dim)(encoder_inputs)
encoder_lstm = LSTM(latent_dim, return_state=True)
encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)
encoder_states = [state_h, state_c]

# Decoder
decoder_inputs = Input(shape=(None,))
decoder_embedding = Embedding(hin_vocab_size, latent_dim)(decoder_inputs)
decoder_lstm = LSTM(latent_dim, return_sequences=True, return_state=True)
decoder_outputs, _, _ = decoder_lstm(decoder_embedding, initial_state=encoder_states)
decoder_dense = Dense(hin_vocab_size, activation="softmax")
decoder_outputs = decoder_dense(decoder_outputs)

# Model
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	-
input_layer_1 (InputLayer)	(None, None)	0	-
embedding (Embedding)	(None, None, 256)	17,408	input_layer[0][0]
embedding_1 (Embedding)	(None, None, 256)	33,536	input_layer_1[0][0]
lstm (LSTM)	[(None, 256), (None, 256), (None, 256)]	525,312	embedding[0][0]
lstm_1 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	525,312	embedding_1[0][0], lstm[0][1], lstm[0][2]
dense (Dense)	(None, None, 131)	33,667	lstm_1[0][0]

Total params: 1.135.235 (4.33 MB)

```
model.fit([encoder_input_data, decoder_input_data], decoder_target_data,
          batch_size=64,
          epochs=10,
          validation_split=0.2
)
```

```
# Save the model for later use
model.save("eng-hin-translation-model.h5")
```

Epoch 1/10
125/125 ————— 15s 84ms/step - accuracy: 0.8954 - loss: 0.6580 - val_accuracy: 0.9642 - val_loss: 0.1550
Epoch 2/10
125/125 ————— 17s 73ms/step - accuracy: 0.9456 - loss: 0.2273 - val_accuracy: 0.9680 - val_loss: 0.1307
Epoch 3/10
125/125 ————— 10s 74ms/step - accuracy: 0.9536 - loss: 0.1815 - val_accuracy: 0.9693 - val_loss: 0.1206
Epoch 4/10
125/125 ————— 10s 74ms/step - accuracy: 0.9584 - loss: 0.1611 - val_accuracy: 0.9701 - val_loss: 0.1152
Epoch 5/10
125/125 ————— 9s 68ms/step - accuracy: 0.9629 - loss: 0.1430 - val_accuracy: 0.9712 - val_loss: 0.1107
Epoch 6/10
125/125 ————— 11s 72ms/step - accuracy: 0.9658 - loss: 0.1310 - val_accuracy: 0.9722 - val_loss: 0.1058
Epoch 7/10
125/125 ————— 10s 69ms/step - accuracy: 0.9680 - loss: 0.1221 - val_accuracy: 0.9734 - val_loss: 0.1021
Epoch 8/10
125/125 ————— 11s 73ms/step - accuracy: 0.9711 - loss: 0.1111 - val_accuracy: 0.9741 - val_loss: 0.1004
Epoch 9/10
125/125 ————— 9s 71ms/step - accuracy: 0.9729 - loss: 0.1034 - val_accuracy: 0.9747 - val_loss: 0.1007
Epoch 10/10
125/125 ————— 10s 73ms/step - accuracy: 0.9756 - loss: 0.0941 - val_accuracy: 0.9754 - val_loss: 0.0977
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered

```
# Encoder Model for Inference
encoder_model = Model(encoder_inputs, encoder_states)

# Decoder Model for Inference
decoder_state_input_h = Input(shape=(latent_dim,))
decoder_state_input_c = Input(shape=(latent_dim,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]

decoder_outputs, state_h, state_c = decoder_lstm(
    decoder_embedding, initial_state=decoder_states_inputs
)
decoder_states = [state_h, state_c]
decoder_outputs = decoder_dense(decoder_outputs)

decoder_model = Model(
    [decoder_inputs] + decoder_states_inputs,
    [decoder_outputs] + decoder_states
)
```

```
# Reverse-lookup token index to decode sequences back to text
reverse_eng_char_index = {index: char for char, index in eng_tokenizer.word_index.items()}
reverse_hin_char_index = {index: char for char, index in hin_tokenizer.word_index.items()}
```

```
def decode_sequence(input_seq):
    # Encode the input as state vectors.
    states_value = encoder_model.predict(input_seq)
```

```
    # Generate empty target sequence of length 1.
```

```

target_seq = np.zeros((1, 1))
target_seq[0, 0] = hin_tokenizer.word_index['\t']

# Generate decoded sentence
decoded_sentence = ''

while True:
    output_tokens, h, c = decoder_model.predict([target_seq] + states_value)

    # Sample a token
    sampled_token_index = np.argmax(output_tokens[0, -1, :])
    sampled_char = reverse_hin_char_index.get(sampled_token_index, '')

    if sampled_char == '\n' or len(decoded_sentence) > max_decoder_seq_length:
        break

    decoded_sentence += sampled_char

    # Update the target sequence
    target_seq = np.zeros((1, 1))
    target_seq[0, 0] = sampled_token_index

    # Update states
    states_value = [h, c]

return decoded_sentence

def translate(sentence):
    sequence = eng_tokenizer.texts_to_sequences([sentence])
    sequence = pad_sequences(sequence, maxlen=max_encoder_seq_length, padding='post')
    translation = decode_sequence(sequence)
    return translation

# Test
input_sentence = "IIIT Kota is good"
translated_sentence = translate(input_sentence.lower())
print(f"English: {input_sentence}")
print(f"Hindi: {translated_sentence}")

```



1/1	0s 49ms/step
1/1	0s 52ms/step
1/1	0s 52ms/step
1/1	0s 57ms/step
1/1	0s 57ms/step
1/1	0s 138ms/step
1/1	0s 63ms/step
1/1	0s 51ms/step
1/1	0s 54ms/step
1/1	0s 64ms/step
1/1	0s 50ms/step
1/1	0s 32ms/step
1/1	0s 34ms/step
1/1	0s 36ms/step
1/1	0s 35ms/step
1/1	0s 35ms/step
1/1	0s 42ms/step
1/1	0s 35ms/step
1/1	0s 44ms/step
1/1	0s 34ms/step
1/1	0s 32ms/step
1/1	0s 33ms/step
1/1	0s 32ms/step
1/1	0s 34ms/step
1/1	0s 33ms/step
1/1	0s 33ms/step
1/1	0s 34ms/step
1/1	0s 33ms/step
1/1	0s 33ms/step
1/1	0s 37ms/step
1/1	0s 35ms/step
1/1	0s 36ms/step
1/1	0s 33ms/step
1/1	0s 33ms/step
1/1	0s 32ms/step

English: IIIT Kota is good
Hindi: प्रोजेक्ट बार के सार्ड को दिखाएं